Отчёт по исследовательской работе: Разработка и оптимизация RAG-системы для технической документации ViPNet Coordinator HW 5

1. Введение

Целью данной работы являлась разработка и исследование методов повышения качества Retrieval-Augmented Generation (RAG) системы для автоматизации обработки запросов по технической документации продукта «ViPNet Coordinator HW 5». В рамках исследования были реализованы две архитектуры RAG: базовая (baseline) и система на базе фреймворка LangChain с использованием векторной базы данных Qdrant. Проведена сравнительная оценка качества обеих систем с применением метрик фреймворка RAGAS.

Работа разделена на четыре ключевых этапа:

1. Загрузка и предобработка данных из PDF-документов.
2. Реализация RAG-систем различной сложности.
3. Выбор метрик качества и разработка функций для их вычисления.
4. Сравнительный анализ полученных результатов.

2. Загрузка и предобработка данных

Источником знаний для системы послужил архив с технической документацией, состоящий из шести PDF-файлов (руководства по подготовке, настройке CLI/WEB, справочник и др.).

2.1. Извлечение текста

Для извлечения текстового содержимого из PDF-файлов использовалась библиотека `pdfplumber`, которая показала высокую точность при работе со сложными многостраничными документами.

```
def text_from_pdf(data_path):
    docs = []
    for file in os.listdir(data_path):
        with pdfplumber.open(os.path.join(data_path, file)) as pdf:
            all_text = ""
            for page in pdf.pages:
                all_text += page.extract_text() + "\n"
```

```
            docs.append({'file_name': file, 'text': all_text})
    return docs
```

2.2. Очистка и нормализация текста

Сырой текст из PDF содержал большое количество шума: оглавления, колонтитулы, номера страниц, повторяющуюся контактную информацию и символы товарных знаков. Для повышения качества последующей векторизации была разработана функция комплексной очистки `clean_text_for_rag`. В основе функции лежит набор регулярных выражений, направленных на удаление неинформативных элементов, характерных для технической документации.

```
def clean_text_for_rag(input_data: List[Dict[str, Any]]) -> List[Dict[str,
Any]]:
    # ... удаление оглавления, колонтитулов, копирайта, контактов ...
    text = re.sub(r'© АО «ИнфоТеКС», \d{4}', '', text)
    text = re.sub(r'Телефон:.*?(?=\n|$)', '', text)
    # ... фильтрация строк с моделями устройств ...
    # ... удаление маркеров списков и лишних пробелов ...
    return cleaned_data
```

Применение очистки позволило значительно сократить объем обрабатываемых данных (от нескольких сотен до десятков тысяч символов на документ), удалив нерелевантную информацию и снизив риск попадания шума в контекст LLM.

3. Реализация RAG-систем

3.1. Базовая система (Baseline)

Первая система представляет собой классическую реализацию RAG "с нуля" с использованием минимального набора инструментов.

- **Модель эмбеддингов:** `Qwen/Qwen3-Embedding-0.6B` от Hugging Face.
- **Генеративная модель:** `Qwen/Qwen3-0.6B` с использованием пайплайна Hugging Face.
- **Разбиение текста:** Простое символьное разбиение на чанки фиксированного размера (1000 символов) с перекрытием (200 символов) для сохранения контекста между фрагментами.
- **Векторная БД:** Индекс на основе FAISS (`IndexFlatL2`), хранящийся в оперативной памяти и на диске.

После создания эмбеддингов для всех чанков и построения FAISS-индекса, был реализован пайплайн обработки запроса: поиск релевантных чанков -> формирование промпта -> генерация ответа моделью -> постобработка для извлечения чистого ответа.

```
def system_pipline(query, index, texts, top_k=5, max_context_size=1000):
    # 1. Поиск релевантных чанков
    relevant_chunks, distances = search_relevant_chunks(query, index, texts,
top_k)
    # 2. Генерация ответа
    generated_output = generate_answer(query, relevant_chunks,
max_context_size)
    # 3. Постобработка
    answer = postprocess_answer(generated_output)
    return answer, relevant_chunks
```

3.2. Система на базе LangChain

Вторая система была построена с использованием фреймворка LangChain, что позволило унифицировать компоненты и упростить интеграцию с современными инструментами.

- **Загрузка и разбиение:** Использован `DirectoryLoader` для пакетной загрузки PDF и `RecursiveCharacterTextSplitter` для интеллектуального разбиения текста с сохранением целостности предложений и абзацев.
- **Модель эмбеддингов:** `HuggingFaceEmbeddings` (модель `Qwen/Qwen3-Embedding-0.6B`), интегрированная с LangChain.
- **Генеративная модель:** `HuggingFacePipeline` для обертки ранее созданного пайплайна `Qwen/Qwen3-0.6B`.
- **Векторная БД:** Qdrant. В отличие от FAISS, Qdrant обеспечивает персистентное хранение данных на диске (`QdrantClient(path="./qdrant_data")`) и поддерживает расширенные возможности поиска.
- **Ретривер и QA-цепочка:** Создан ретривер на основе Qdrant, который возвращает топ-5 наиболее релевантных чанков. Вся логика "поиск-генерация" объединена в цепочку `RetrievalQA` с кастомным промптом.

```
# Создание цепочки RetrievalQA
qa_chain = RetrievalQA.from_chain_type(
    llm=langchain_llm,
    chain_type="stuff",
    retriever=retriever,
    chain_type_kwargs={"prompt": prompt},
    return_source_documents=True,
)
```

4. Метрики и методология оценки

Для количественной оценки качества RAG-систем был использован фреймворк **RAGAS**. Оценка проводилась по четырем ключевым метрикам, охватывающим как компонент поиска (ретривер), так и компонент генерации:

1. **Контекстная точность (Context Precision):** Оценивает, насколько релевантны извлеченные чанки по отношению к запросу. Высокое значение означает, что в топе результатов находится мало шума.
2. **Контекстная полнота (Context Recall):** Измеряет, была ли извлечена вся информация, необходимая для ответа на вопрос. Для её расчета требуются эталонные ответы (ground truth).
3. **Фактологичность (Faithfulness):** Оценивает, насколько сгенерированный ответ соответствует фактам, содержащимся в извлеченном контексте, и не содержит ли "галлюцинаций".
4. **Корректность ответа (Answer Correctness):** Комплексная метрика, оценивающая, насколько ответ модели совпадает с эталонным по смыслу и фактам.

4.1. Генерация тестового набора данных

Для проведения объективной оценки был сгенерирован тестовый набор данных (test set) из 20 вопросов и эталонных ответов на основе загруженных документов. Для этого использовался встроенный инструмент `TestsetGenerator` из RAGAS. Несмотря на возникавшие ошибки парсинга при генерации, полученный набор был сохранен и использован для дальнейшей валидации.

```
generator = TestsetGenerator.from_langchain(llm=llm,
embedding_model=embeddings)
testset = generator.generate_with_langchain_docs(documents=documents,
testset_size=20)
```

4.2. Процедура оценки

Из-за изменений в архитектуре RAGAS, требующих использования LLM определенного типа (`InstructorBaseRagasLLM`), для оценки была развернута локальная инфраструктура **Ollama**. Модель `qwen3:0.6b` , аналогичная используемым в системах, была запущена через Ollama и интегрирована с RAGAS через фабрику `llm_factory` . Оценка проводилась на сохраненных датасетах с ответами и контекстами от обеих систем.

5. Результаты и их анализ

В результате проведенных экспериментов были получены следующие значения метрик для обеих систем (при использовании `qwen3:0.6b` в качестве "судьи"):

Метрика	Baseline	LangChain	Интерпретация
Context Precision	0.8993	0.9603	LangChain с Qdrant обеспечивает более точный поиск, извлекая меньше шума.
Context Recall	0.9330	0.9310	Обе системы извлекают практически всю необходимую информацию. Разница в пределах погрешности.
Faithfulness	0.9453	0.9464	Генерации обеих моделей в высокой степени соответствуют извлеченному контексту, галлюцинации минимальны.
Answer Correctness	0.6157	0.5689	Ответы модели лишь частично совпадают с эталонными. LangChain-система показала более низкий результат, что может быть связано с форматированием промпта или стратегией объединения контекстов.

Выводы по результатам:

- **Ретривер:** Обе системы демонстрируют высокое качество работы ретривера (метрики Context Precision/Recall > 0.9). Модуль поиска успешно справляется с задачей нахождения релевантных фрагментов. При этом система на LangChain с Qdrant показывает лучшую точность (Precision), что говорит о преимуществе используемого векторного хранилища и поиска путем косинусной близости.
- **Генератор:** Основной "бутылочной горлышком" является генеративная модель. Низкие значения метрики Answer Correctness (0.57-0.62) при высокой фактологичности (Faithfulness) указывают на то, что модель **Qwen3-0.6B** часто отвечает неполно или не совсем так, как ожидается в эталонном ответе. Она правильно использует факты из контекста, но плохо структурирует ответ или упускает важные детали.

6. Использование

Структура проекта

- `README.md` — основной файл документации с описанием проекта, методологии и результатов
- `requirements.txt` — зависимости проекта
- `research/` — папка с исследовательскими материалами
  - `datasets/` — директория для хранения датасетов
  - `research.ipynb` — исследовательский ноутбук с анализом и экспериментами
- `app/` — основное приложение

- `setup.py` — скрипт установки и запуска приложения
- `systems/` — модули реализации RAG-систем
  - `PDFPlumberLoader.py` — загрузчик PDF-документов
  - `BaseLineSystem.py` — реализация базовой RAG-системы
  - `LangChainSystem.py` — реализация RAG-системы на базе LangChain
  - `VectorDBBuilder.py` — вспомогательный модуль для построения векторной БД

Запуск приложения

Для запуска приложения необходимо:

1. Установить зависимости:
   ```
   pip install -r requirements.txt
   ```
2. Запустить приложение через setup.py:
   ```
   python app/setup.py
   ```

Для корректной работы приложения требуется запущенный Docker-контейнер с Qdrant:

```
docker run -p 6333:6333 qdrant/qdrant
```

А также GPU с поддержкой CUDA для ускорения вычислений.

7. Заключение

В ходе работы были успешно разработаны и протестированы две RAG-системы для работы с технической документацией. Исследование показало, что использование фреймворка LangChain в сочетании с векторной БД Qdrant позволяет создать более гибкую и точную систему по сравнению с самописной реализацией на FAISS, особенно в части точности извлечения контекста (Context Precision вырос с 0.899 до 0.960).

Ключевым ограничением текущей реализации является используемая генеративная модель с 0.6 миллиардами параметров. Несмотря на высокую скорость работы, её способность генерировать полные и корректные ответы ограничена, что подтверждается метрикой Answer Correctness.

Рекомендации по улучшению системы:

1. **Замена генеративной модели:** Наиболее эффективным улучшением станет замена текущей модели на более мощную (в рамках ограничений, например, на 7-8B параметров), что должно значительно повысить корректность и полноту ответов.

2. **Оптимизация промпта:** Уточнение инструкций в системном промпте может помочь модели лучше структурировать ответ, даже при ограниченном размере.
3. **Улучшение качества тестовых данных:** Повторная генерация тестового набора с более тщательной валидацией может снизить влияние шума в эталонных ответах на итоговую оценку.